

*S11BLH21- Programming in python*



**SATHYABAMA**

INSTITUTE OF SCIENCE AND TECHNOLOGY

(DEEMED TO BE UNIVERSITY)

Accredited "A" Grade by NAAC | 12B Status by UGC | Approved by AICTE

[www.sathyabama.ac.in](http://www.sathyabama.ac.in)

**SCHOOL OF COMPUTING**

**DEPARTMENT OF COMPUTER SCIENCE AND  
ENGINEERING**

## **UNIT-V – S11BLH21- Programming in python**

### **DATABASE AND NETWORK**

Database (using NoSQL): Connector Module –Cursor – Statements - Exceptions in database. Network connectivity: Socket module - Client – Server –Email – URL Access.

## DATABASE AND NETWORK

Data is very important for any organization to continue its operations. The data may be related to employees in the organization or the operational data like products information, raw material prices, sales information, profits and losses. Without data, no organization will survive. Hence, data is very important and it should never be lost.

### *DATABASE MANAGEMENT SYSTEM (DBMS)*

To store data, a file or database can be used. A file stores data in the secondary storage device like hard disk, either in the text format or binary format.

A database represents collection of data. Data is stored in the database. Once the data is stored in the database, various operations can be performed on the data. For example, modifying the existing data, deleting the unwanted data, or retrieving the data from the database and etc. To perform such operations, a database comes with software. This is called a database management system.

DBMS= Database + Software to manage the data

Example DBMS are MySQL, Oracle, Sybase,, SQL server etc.

Types of databases used with Python

## *S11BLH21- Programming in python*

### *1. Database support*

- SQL
- NoSQL

As more and more data become available as unstructured or semi-structured, the need of managing them through NoSql database increases. Python can also interact with NoSQL databases in a similar way as it interacts with Relational databases. In this chapter we will use python to interact with MongoDB as a NoSQL database.

### *MONGO DB*

MongoDB stores data in JSON-like documents, which makes the database very flexible and scalable.

WheretouseMongoDB?

- Big Data
- Content Management and Delivery
- Mobile and Social Infrastructure
- User Data Management
- Data Hub

download a free MongoDB database at <https://www.mongodb.com>.

### *PyMongo*

Python needs a MongoDB driver to access the MongoDB database.

## *S11BLH21- Programming in python*

In this tutorial we will use the MongoDB driver "PyMongo".

We recommend that you use PIP to install "PyMongo".

PIP is most likely already installed in your Python environment.

Navigate your command line to the location of PIP, and type the following:

Download and install "PyMongo":

```
C:\Users\Your Name\AppData\Local\Programs\Python\Python36-32\Scripts>python  
-m pip install pymongo
```

Now you have downloaded and installed a mongoDB driver.

WheretouseMongoDB?

- └ Big Data
- └ Content Management and Delivery
- └ Mobile and Social Infrastructure
- └ User Data Management
- └ Data Hub

### *Test PyMongo*

To test if the installation was successful, or if you already have "pymongo" installed, create a Python page with the following content:

demo\_mongodb\_test.py:

```
import pymongo
```

## *S11BLH21- Programming in python*

### Creating a Database

To create a database in MongoDB, start by creating a MongoClient object, then specify a connection URL with the correct ip address and the name of the database you want to create.

MongoDB will create the database if it does not exist, and make a connection to it.

### *Example*

Create a database called mydatabase

### *Program*

```
import pymongo

myclient      =      pymongo.MongoClient("mongodb://localhost:27017/")

mydb = myclient["mydatabase"]
```

MongoDB waits until you have created a collection (table), with at least one document (record) before it actually creates the database (and collection).

### ***4.2 CREATING A COLLECTION***

To create a collection in MongoDB, use database object and specify the name of the collection you want to create.

MongoDB will create the collection if it does not exist.

## *S11BLH21- Programming in python*

### *Program*

```
import pymongo

myclient = pymongo.MongoClient("mongodb://localhost:27017/")
mydb = myclient["mydatabase"]
mycol = mydb["customers"]
```

MongoDB waits until you have inserted a document before it actually creates the collection.

### *4.3.1 Python MongoDB Insert Document*

#### *Insert Into Collection*

To insert a record, or *document* as it is called in MongoDB, into a collection, we use the `insert_one()` method.

The first parameter of the `insert_one()` method is a dictionary containing the name(s) and value(s) of each field in the document you want to insert.

#### *Example*

Insert a record in the “Customers” Collection:

```
import pymongo

myclient = pymongo.MongoClient("mongodb://localhost:27017/")
mydb = myclient["mydatabase"]
mycol = mydb["customers"]
```

## *S11BLH21- Programming in python*

```
mydict = { "name": "John", "address": "Highway 37" }  
x = mycol.insert_one(mydict)
```

### **Insert Multiple Documents**

To insert multiple documents into a collection in MongoDB, we use the `insert_many()` method.

The first parameter of the `insert_many()` method is a list containing dictionaries with the data you want to insert:

```
import pymongo  
myclient = pymongo.MongoClient("mongodb://localhost:27017/")  
mydb = myclient["mydatabase"]  
mycol = mydb["customers"]  
  
mylist = [  
    { "name": "Amy", "address": "Apple st 652"},  
    { "name": "Hannah", "address": "Mountain 21"},  
    { "name": "Michael", "address": "Valley 345"},  
    { "name": "Sandy", "address": "Ocean blvd 2"},  
    { "name": "Betty", "address": "Green Grass 1"},  
    { "name": "Richard", "address": "Sky st 331"},  
    { "name": "Susan", "address": "One way 98"},  
    { "name": "Vicky", "address": "Yellow Garden 2"},  
    { "name": "Ben", "address": "Park Lane 38"},  
    { "name": "William", "address": "Central st 954"},  
]
```

## *S11BLH21- Programming in python*

```
{ "name": "Chuck", "address": "Main Road 989"},  
{ "name": "Viola", "address": "Sideway 1633"}
```

```
x = mycol.insert_many(mylist)
```

### *Python MongoDB Find*

In MongoDB we use the **find** and **findOne** methods to find data in a collection.

Just like the **SELECT** statement is used to find data in a table in a MySQL database.

### **Find One**

To select data from a collection in MongoDB, we can use the `find_one()` method.

The `find_one()` method returns the first occurrence in the selection.

Example

Find the first document in the customers collection:

```
import pymongo  
  
myclient = pymongo.MongoClient("mongodb://localhost:27017/")  
mydb = myclient["mydatabase"]  
mycol = mydb["customers"]  
  
x = mycol.find_one()
```



## *S11BLH21- Programming in python*

```
print(x)
```

Output

```
{'_id': 1, 'name': 'John', 'address': 'Highway37'}Find All
```

To select data from a table in MongoDB, we can also use the find() method.

The find() method returns all occurrences in the selection.

The first parameter of the find() method is a query object. In this example we use an empty query object, which selects all documents in the collection.

Example

Return all documents in the "customers" collection, and print each document:

```
import pymongo
```

```
myclient = pymongo.MongoClient("mongodb://localhost:27017/")
```

```
mydb = myclient["mydatabase"]
```

```
mycol = mydb["customers"]
```

```
for x in mycol.find():
```

```
print(x)
```

## *S11BLH21- Programming in python*

```
{'_id': 1, 'name': 'John', 'address': 'Highway37'}  
{'_id': 2, 'name': 'Peter', 'address': 'Lowstreet 27'}  
{'_id': 3, 'name': 'Amy', 'address': 'Apple st 652'}  
{'_id': 4, 'name': 'Hannah', 'address': 'Mountain 21'}  
{'_id': 5, 'name': 'Michael', 'address': 'Valley 345'}  
{'_id': 6, 'name': 'Sandy', 'address': 'Ocean blvd 2'}  
{'_id': 7, 'name': 'Betty', 'address': 'Green Grass 1'}  
{'_id': 8, 'name': 'Richard', 'address': 'Sky st 331'}  
{'_id': 9, 'name': 'Susan', 'address': 'One way 98'}  
{'_id': 10, 'name': 'Vicky', 'address': 'Yellow Garden 2'}  
{'_id': 11, 'name': 'Ben', 'address': 'Park Lane 38'}  
{'_id': 12, 'name': 'William', 'address': 'Central st 954'}  
{'_id': 13, 'name': 'Chuck', 'address': 'Main Road 989'}  
{'_id': 14, 'name': 'Viola', 'address': 'Sideway 1633'}
```

### *Filter the Result*

- When finding documents in a collection, you can filter the result by using a queryobject.
- The first argument of the find() method is a query object, and is used to limit the search.

### *Example*

Find document(s) with the address "Park Lane 38":

```
import pymongo  
myclient = pymongo.MongoClient("mongodb://localhost:27017/")  
mydb = myclient["mydatabase"]
```

## *S11BLH21- Programming in python*

```
mycol = mydb["customers"]
myquery = { "address": "Park Lane 38" }

mydoc = mycol.find(myquery)
for x in mydoc:
    print(x)
```

output

```
{'_id': 11, 'name': 'Ben', 'address': 'Park Lane 38'}
```

### *Example*

Find documents where the address starts with the letter "S" or higher:

```
import pymongo
myclient = pymongo.MongoClient("mongodb://localhost:27017/")
mydb = myclient["mydatabase"]
mycol = mydb["customers"]
myquery = { "address": { "$gt": "S" } }
mydoc = mycol.find(myquery)
for x in mydoc:
    print(x)
```

## *S11BLH21- Programming in python*

### *Output*

```
{'_id': 5, 'name': 'Michael', 'address': 'Valley 345'}  
{'_id': 8, 'name': 'Richard', 'address': 'Sky st 331'}  
{'_id': 10, 'name': 'Vicky', 'address': 'Yellow  
Garden 2'}  
{'_id': 14, 'name': 'Viola', 'address': 'Sideway1633'}
```

## *S11BLH21- Programming in python*

### **Return Only Some Fields**

The second parameter of the find() method is an object describing which fields to include in the result.

This parameter is optional, and if omitted, all fields will be included in the result.

### *Example*

Return only the names and addresses, not the \_ids:

```
import pymongo
myclient = pymongo.MongoClient("mongodb://localhost:27017/")
mydb = myclient["mydatabase"]

mycol = mydb["customers"]

for x in mycol.find({}, {"_id": 0, "name": 1, "address": 1}):
    print(x)
```

### *Output*

```
{'name': 'John', 'address': 'Highway37'}
{'name': 'Peter', 'address': 'Lowstreet 27'}
{'name': 'Amy', 'address': 'Apple st 652'}
{'name': 'Hannah', 'address': 'Mountain 21'}
{'name': 'Michael', 'address': 'Valley 345'}
{'name': 'Sandy', 'address': 'Ocean blvd 2'}
{'name': 'Betty', 'address': 'Green Grass 1'}
{'name': 'Richard', 'address': 'Sky st 331'}
```

## *S11BLH21- Programming in python*

```
{'name': 'Susan', 'address': 'One way 98'}  
{'name': 'Vicky', 'address': 'Yellow Garden 2'}  
{'name': 'Ben', 'address': 'Park Lane 38'}  
{'name': 'William', 'address': 'Central st 954'}  
{'name': 'Chuck', 'address': 'Main Road 989'}  
{'name': 'Viola', 'address': 'Sideway 1633'}
```

### *Sort the Result*

Use the sort() method to sort the result in ascending or descending order.

The sort() method takes one parameter for "fieldname" and one parameter for "direction" (ascending is the default direction).

### *Example*

Sort the result alphabetically by name:

```
import pymongo  
  
myclient = pymongo.MongoClient("mongodb://localhost:27017/")  
mydb = myclient["mydatabase"]  
mycol = mydb["customers"]  
  
mydoc = mycol.find().sort("name")  
  
for x in mydoc:  
    print(x)
```

OUTPUT

## *S11BLH21- Programming in python*

```
{'_id': 3, 'name': 'Amy', 'address': 'Apple st 652'}
{'_id': 11, 'name': 'Ben', 'address': 'Park Lane 38'}
{'_id': 7, 'name': 'Betty', 'address': 'Green Grass 1'}
{'_id': 13, 'name': 'Chuck', 'address': 'Main Road 989'}
{'_id': 4, 'name': 'Hannah', 'address': 'Mountain 21'}
{'_id': 1, 'name': 'John', 'address': 'Highway37'}
{'_id': 5, 'name': 'Michael', 'address': 'Valley 345'}
{'_id': 2, 'name': 'Peter', 'address': 'Lowstreet 27'}
{'_id': 8, 'name': 'Richard', 'address': 'Sky st 331'}
{'_id': 6, 'name': 'Sandy', 'address': 'Ocean blvd 2'}
{'_id': 9, 'name': 'Susan', 'address': 'One way 98'}
{'_id': 10, 'name': 'Vicky', 'address': 'Yellow Garden 2'}
{'_id': 14, 'name': 'Viola', 'address': 'Sideway 1633'}
{'_id': 12, 'name': 'William', 'address': 'Central st 954'}
```

### **Sort Descending**

Use the value -1 as the second parameter to sort descending.

```
sort("name", 1) #ascending
sort("name", -1) #descending
```

### *Example*

Sort the result reverse alphabetically by name:

```
import pymongo

myclient = pymongo.MongoClient("mongodb://localhost:27017/")
mydb = myclient["mydatabase"]
mycol = mydb["customers"]
```

## *S11BLH21- Programming in python*

```
mydoc = mycol.find().sort("name", -1)

for x in mydoc:
    print(x)
```

### *Output*

```
{'_id': 12, 'name': 'William', 'address': 'Central st 954'}
{'_id': 14, 'name': 'Viola', 'address': 'Sideway 1633'}
{'_id': 10, 'name': 'Vicky', 'address': 'Yellow Garden 2'}
{'_id': 9, 'name': 'Susan', 'address': 'One way 98'}
{'_id': 6, 'name': 'Sandy', 'address': 'Ocean blvd 2'}
{'_id': 8, 'name': 'Richard', 'address': 'Sky st 331'}
{'_id': 2, 'name': 'Peter', 'address': 'Lowstreet 27'}
{'_id': 5, 'name': 'Michael', 'address': 'Valley 345'}
{'_id': 1, 'name': 'John', 'address': 'Highway37'}
{'_id': 4, 'name': 'Hannah', 'address': 'Mountain 21'}
{'_id': 13, 'name': 'Chuck', 'address': 'Main Road 989'}
{'_id': 7, 'name': 'Betty', 'address': 'Green Grass 1'}
{'_id': 11, 'name': 'Ben', 'address': 'Park Lane 38'}
{'_id': 3, 'name': 'Amy', 'address': 'Apple st 652'}
```

### *Python MongoDB Delete Document*

To delete one document, we use the `delete_one()` method.

The first parameter of the `delete_one()` method is a query object defining which document to delete.

**Note:** If the query finds more than one document, only the first occurrence is deleted.



## *S11BLH21- Programming in python*

### *Example*

Delete the document with the address "Mountain 21":

```
import pymongo

myclient = pymongo.MongoClient("mongodb://localhost:27017/")
mydb = myclient["mydatabase"]
mycol = mydb["customers"]

myquery = { "address": "Mountain 21" }

mycol.delete_one(myquery)
```

### **Delete Many Documents**

To delete more than one document, use the delete\_many() method.

The first parameter of the delete\_many() method is a query object defining which documents to delete.

### *Example*

Delete all documents where the address starts with the letter S:

```
import pymongo

myclient = pymongo.MongoClient("mongodb://localhost:27017/")
mydb = myclient["mydatabase"]
mycol = mydb["customers"]
```

## *S11BLH21- Programming in python*

```
myquery = { "address": { "$regex": "^S" } }

x = mycol.delete_many(myquery)

print(x.deleted_count, " documents deleted.")

output

2 documents deleted.
```

### **Delete All Documents in a Collection**

To delete all documents in a collection, pass an empty query object to the `delete_many()` method:

#### *Example*

Delete all documents in the "customers" collection:

```
import pymongo

myclient = pymongo.MongoClient("mongodb://localhost:27017/")
mydb = myclient["mydatabase"]
mycol = mydb["customers"]

x = mycol.delete_many({})

print(x.deleted_count, " documents deleted.")
```

## *S11BLH21- Programming in python*

*Output:*

**11 documents deleted**

### *4.3.5 Python MongoDB Drop Collection*

#### *Delete Collection*

You can delete a table, or collection as it is called in MongoDB, by using the drop() method.

*Example*

Delete the "customers" collection:

```
import pymongo

myclient = pymongo.MongoClient("mongodb://localhost:27017/")
mydb = myclient["mydatabase"]
mycol = mydb["customers"]

mycol.drop()
```

The drop() method returns true if the collection was dropped successfully, and false if the collection does not exist.

## *S11BLH21- Programming in python*

### *Python MongoDB Update*

You can update a record, or document as it is called in MongoDB, by using the `update_one()` method.

The first parameter of the `update_one()` method is a query object defining which document to update.

**Note:** If the query finds more than one record, only the first occurrence is updated.

### *Example*

Change the address from "Valley 345" to "Canyon 123":

```
import pymongo

myclient = pymongo.MongoClient("mongodb://localhost:27017/")
mydb = myclient["mydatabase"]
mycol = mydb["customers"]

myquery = { "address": "Valley 345" }
newvalues = { "$set": { "address": "Canyon 123" } }

mycol.update_one(myquery, newvalues)

#print "customers" after the update:
for x in mycol.find():
    print(x)
```

## *S11BLH21- Programming in python*

### *OUTPUT*

```
{'_id': 1, 'name': 'John', 'address': 'Highway37'}  
{'_id': 2, 'name': 'Peter', 'address': 'Lowstreet 27'}  
{'_id': 3, 'name': 'Amy', 'address': 'Apple st 652'}  
{'_id': 4, 'name': 'Hannah', 'address': 'Mountain 21'}  
{'_id': 5, 'name': 'Michael', 'address': 'Canyon 123'}  
{'_id': 6, 'name': 'Sandy', 'address': 'Ocean blvd 2'}  
{'_id': 7, 'name': 'Betty', 'address': 'Green Grass 1'}  
{'_id': 8, 'name': 'Richard', 'address': 'Sky st 331'}  
{'_id': 9, 'name': 'Susan', 'address': 'One way 98'}  
{'_id': 10, 'name': 'Vicky', 'address': 'Yellow Garden 2'}  
{'_id': 11, 'name': 'Ben', 'address': 'Park Lane 38'}  
{'_id': 12, 'name': 'William', 'address': 'Central st 954'}  
{'_id': 13, 'name': 'Chuck', 'address': 'Main Road 989'}  
{'_id': 14, 'name': 'Viola', 'address': 'Sidewalk y'}
```

### **Update Many**

To update *all* documents that meets the criteria of the query, use the `update_many()` method.

### *Example*

Update all documents where the address starts with the letter "S":

```
import pymongo  
  
myclient = pymongo.MongoClient("mongodb://localhost:27017/")  
mydb = myclient["mydatabase"]  
mycol = mydb["customers"]
```

## *S11BLH21- Programming in python*

```
myquery = { "address": { "$regex": "^S" } }  
newvalues = { "$set": { "name": "Minnie" } } x  
  
= mycol.update_many(myquery, newvalues)  
  
print(x.modified_count, "documents updated.")
```

### *Output*

**2 documents updated.**

### *Python MongoDB Limit*

To limit the result in MongoDB, we use the limit() method.

The limit() method takes one parameter, a number defining how many documents to return.

Consider you have a "customers" collection:

```
{'_id': 1, 'name': 'John', 'address': 'Highway37'}  
{'_id': 2, 'name': 'Peter', 'address': 'Lowstreet 27'}  
{'_id': 3, 'name': 'Amy', 'address': 'Apple st 652'}  
{'_id': 4, 'name': 'Hannah', 'address': 'Mountain 21'}  
{'_id': 5, 'name': 'Michael', 'address': 'Valley 345'}  
{'_id': 6, 'name': 'Sandy', 'address': 'Ocean blvd 2'}  
{'_id': 7, 'name': 'Betty', 'address': 'Green Grass 1'}  
{'_id': 8, 'name': 'Richard', 'address': 'Sky st 331'}  
{'_id': 9, 'name': 'Susan', 'address': 'One way 98'}  
{'_id': 10, 'name': 'Vicky', 'address': 'Yellow Garden 2'}  
{'_id': 11, 'name': 'Ben', 'address': 'Park Lane 38'}
```

## *S11BLH21- Programming in python*

```
{'_id': 12, 'name': 'William', 'address': 'Central st 954'}  
{'_id': 13, 'name': 'Chuck', 'address': 'Main Road 989'}  
{'_id': 14, 'name': 'Viola', 'address': 'Sidewalky'}
```

### *Example*

Limit the result to only return 5 documents:

```
import pymongo  
  
myclient = pymongo.MongoClient("mongodb://localhost:27017/")  
mydb = myclient["mydatabase"]  
mycol = mydb["customers"]  
  
myresult = mycol.find().limit(5)  
  
#print the result:  
for x in myresult:  
    print(x)
```

### *OUTPUT*

```
{'_id': 1, 'name': 'John', 'address': 'Highway37'}  
{'_id': 2, 'name': 'Peter', 'address': 'Lowstreet 27'}  
{'_id': 3, 'name': 'Amy', 'address': 'Apple st 652'}  
{'_id': 4, 'name': 'Hannah', 'address': 'Mountain 21'}  
{'_id': 5, 'name': 'Michael', 'address': 'Valley 345'}
```

## *S11BLH21- Programming in python*

### ***CURSOR CLASS***

To work with MySQL in python, connector sub module of mysql module.

```
import mysql.connector;
```

*to establish connection with MySQL database, we use the connect() method of mysql.connector module as:*

```
conn=mysql.connector.connect(host="localhost",database="university",user="root",  
password="****")
```

*The connect() method returns MySQLConnection class object 'conn'.*

The next step is to create cursor class object by calling the cursor() method on „conn“ object as:

```
cursor=con.cursor()
```

*Cursor object is useful to execute SQL commands on the database.*

it is done by execute() method of cursor object.

```
cursor.execute( sql query)
```

```
example: cursor.execute("select * from emptab")
```

*The resultant rows retrieved from the table are stored in cursor object. the result can be fetched using fetchone() or fetchall() methods.*

```
example: row = cursor.fetchone() # get 1 row
```



## *S11BLH21- Programming in python*

```
row = cursor.fetchall() # get all rows
```

*Finally, the connection with MySQL can be closed by closing the cursor and connection objects as:*

```
cursor.close()
```

```
conn.close()
```

*Program: A python program to retrieve and display all rows from the studenttable:*

```
import mysql.connector;
```

```
conn=mysql.connector.connect(host="localhost",database="university",user="root",  
password="****")
```

```
cursor=con.cursor()
```

```
cursor.execute("select * from stutab")
```

```
row = cursor.fetchone()
```

```
while row is not None:
```

```
    print(row)
```

```
    row=cursor.fetchone()
```

```
cursor.close()
```

```
conn.close()
```

## *S11BLH21- Programming in python*

*Output:*

(1001, „Ajay“, 8.5)

(1002, „Alan“, 7.5)

(1001, „Joe“, 9.00)

### *EXCEPTIONS CLASSES*

Interacting with a database is an error prone process, so we must always implement some mechanism to handle errors.

*Built in Exceptions*

| Exception               | Description                                                                                                                                                                 |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Warning</b>          | Used for non-fatal issues. Must subclass StandardError.                                                                                                                     |
| <b>Error</b>            | Base class for errors. Must subclass StandardError.                                                                                                                         |
| <b>InterfaceError</b>   | Used for errors in the database module, not the database itself. Must subclass Error.                                                                                       |
| <b>DatabaseError</b>    | Used for errors in the database. Must subclass Error.                                                                                                                       |
| <b>DataError</b>        | Subclass of DatabaseError that refers to errors in the data.                                                                                                                |
| <b>OperationalError</b> | Subclass of DatabaseError that refers to errors such as the loss of a connection to the database. These errors are generally outside of the control of the Python scripter. |

| Exception               | Description                                                                                                                          |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <b>IntegrityError</b>   | Subclass of DatabaseError for situations that would damage the relational integrity, such as uniqueness constraints or foreign keys. |
| <b>InternalError</b>    | Subclass of DatabaseError that refers to errors internal to the database module, such as a cursor no longer being active.            |
| <b>ProgrammingError</b> | Subclass of DatabaseError that refers to errors such as a bad table name and other things that can safely be blamed on you.          |

## NETWORKING

For a specific purpose if things are connected together, are referred as a NETWORK. A network can be of many types, like a telephone network, television network, computer network or even a people network.

Similarly, a COMPUTER NETWORK is also a kind of setup, where it connects two or more devices to share a range of services and information in the form of e-mails and messages, databases, documents, web-sites, audios and videos, Telephonecalls and video conferences etc among them.

A PROTOCOL is nothing but set of defined rules, which has to be followed by every connected devices across a network to communicate and share information among them. To facilitates End to End communication, a number of protocols worked together to form a Protocol Suites or Stacks

## *S11BLH21- Programming in python*

Some basic Protocols are:

- IP : Internet Protocol
- FTP : File Transfer Protocol
- SMTP : Simple Mail Transfer Protocol
- HTTP : Hyper Text Transfer Protocol

The Network reference models were developed to allow products from different manufacturers to interoperate on a network. A network reference model serves as a blueprint, detailing standards for how protocol communication should occur. The most widely recognized reference models are, the Open Systems Interconnect ( OSI ) Model and Department of Defense ( DoD, also known as TCP/IP ) model.

Network Types are often categorized by their size and functionality. According to the size, the network can be commonly categorized into Three types.

- *LANs (Local Area Networks)*
- **MANs (Metropolitan Area Networks)**
- *WANs (Wide Area Networks)*

An **Internetwork** is a general term describing multiple networks connected together. The Internet is the largest and most well-known internetwork.

Some networks are categorized by their function, as opposed to their size.

For example:

## *S11BLH21- Programming in python*

- **SAN (Storage Area Network):** A SAN provides systems with high-speed, lossless access to high-capacity storage devices.
- **VPN (Virtual Private Network):** A VPN allows for information to be securely sent across a public or unsecure network, such as the Internet. Common uses of a VPN are to connect branch offices or remote users to a main office.

In a network, any connected device is called as **host**. A host can serve as following ways:

- A host can acts as a **Client**, when he is requesting information.
- A host can acts as a **Server**, when he provides information.
- A host can also request and provide information, is called **Peer**.

### ***SOCKET MODULE***

#### *What Are Sockets?*

A socket is a link between two applications that can communicate with one another (either locally on a single machine or remotely between two machines in separate locations).

Basically, sockets act as a communication link between two entities, i.e. a server and a client. A server will give out information being requested by a client. For example, when you visited this page, the browser created a socket and connected to the server.

## *S11BLH21- Programming in python*

### *The socket Module*

In order to create a socket, you use the `socket.socket()` function, and the syntax is as simple as:

```
import socket

s= socket.socket (socket_family, socket_type, protocol=0)
```

Here is the description of the arguments:

- **socket\_family:** Represents the address (and protocol) family. It can be either `AF_UNIX` or `AF_INET`.
- **socket\_type:** Represents the socket type, and can be either `SOCK_STREAM` or `SOCK_DGRAM`.
- **protocol:** This is an optional argument, and it usually defaults to 0.

After obtaining your socket object, you can then create a server or client as desired using the methods available in the socket module.

- `s.recv()` – It receives TCPmessage
- `s.send()` – It transmits TCP message
- `s.recvfrom()` – It receives UDPmessage
- `s.sendto()` – It transmits UDP message
- `s.close()` – It closes socket
- `socket.gethostname()` – It returns thehostname

## *S11BLH21- Programming in python*

### *CREATE A SIMPLE CLIENT*

Before we get started, let's look at the client socket methods available in Python.

```
s= socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
s.connect()Initiates a TCP server connection.
```

*To create a new socket, you first import the socket method of the socket class.*

```
import socket
```

Next, we'll create a stream (TCP) socket as follows:

```
stream_socket = socket.socket( socket.AF_INET, socket.SOCK_STREAM )
```

The AF\_INET argument indicates that you're requesting an Internet Protocol (IP) socket, specifically IPv4. The second argument is the transport protocol type SOCK\_STREAM for TCP sockets. Additionally, you can also create an IPv6 socket by specifying the socket AF\_INET6 argument.

*Specify the server.*

```
server = "localhost"
```

*Specify the port we want to communicate with.*

```
port =80
```

*Connect the socket to the port where the server is listening.*

```
server_address = ((host, port)
```

## *S11BLH21- Programming in python*

```
stream_socket.connect(server_address)
```

It's important to note that the host and port must be a tuple.

*Send a data request to the server:*

```
message = 'message'
```

```
stream_socket.sendall(message)
```

**Get the response from the server:**

```
data = sock.recv(10)
```

```
print data
```

**To close a connected socket, you use the close method:**

```
stream_socket.close()
```

*Below is the full code for the Client/Server.*

```
import socket
```

```
import sys
```

```
# Create a TCP/IP socket
```

```
stream_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
# Define host
```

```
host = 'localhost'
```



## *S11BLH21- Programming in python*

```
# define the communication port

port = 8080

# Connect the socket to the port where the server is listening

server_address = ((host, port))

print "connecting"

stream_socket.connect(server_address)

# Send data

message = 'message'

stream_socket.sendall(message)

# response

data = stream_socket.recv(10)

print data

print 'socket closed'

stream_socket.close()
```

### *BUILD A SIMPLE SERVER*

Now let's take a look at a simple Python server. The following are the socket server methods available in Python.

## *S11BLH21- Programming in python*

```
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

s.bind(): Binds address (hostname, port number) to socket.

s.listen(): Sets up and starts TCP listener.

s.accept(): Accepts TCP client connection.

We will follow the following steps:

- Create a socket.
- Bind the socket to a port.
- Start accepting connections on the socket.

*Here is the server program.*

```
import socket
```

```
import sys
```

```
# Create a TCP/IP socket
```

```
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
# Define host
```

```
host = 'localhost'
```

```
# define the communication port
```

```
port = 8080
```

## *S11BLH21- Programming in python*

```
# Bind the socket to the port

sock.bind((host, port))

# Listen for incoming connections

sock.listen(1)

# Wait for a connection

print 'waiting for a connection'

connection, client = sock.accept()

print client, 'connected'

# Receive the data in small chunks and retransmit it

data = connection.recv(16)

print 'received "%s"' % data

if data:

    connection.sendall(data)

else:

    print 'no data from', client

# Close the connection

connection.close()
```

## *S11BLH21- Programming in python*

The server is now ready for incoming connections.

Now run the client and server programs in separate terminal windows, so they can communicate with each other.

### Server Output

```
$ python server.py  
  
waiting for a connection  
  
('127.0.0.1', 47050) connected  
  
received "message"
```

### Client Output

```
$ python client.py  
  
connecting  
  
message  
  
socket closed
```

## *SENDING EMAIL USING SMTP*

Simple Mail Transfer Protocol (SMTP) is a protocol, which handles sending e-mail and routing e-mail between mail servers.

## *S11BLH21- Programming in python*

Python provides **smtplib** module, which defines an SMTP client session object that can be used to send mail to any Internet machine with an SMTP or ESMTP listener daemon.

Here is a simple syntax to create one SMTP object, which can later be used to send an e-mail –

```
import smtplib
```

```
smtpObj = smtplib.SMTP( [host [, port [, local_hostname]]] )
```

Here is the detail of the parameters –

- **host** – This is the host running your SMTP server. You can specify IP address of the host or a domain name like *tutorialspoint.com*. This is optional argument.
- **port** – If you are providing *host* argument, then you need to specify a port, where SMTP server is listening. Usually this port would be 25.
- **local\_hostname** – If your SMTP server is running on your local machine, then you can specify just *localhost* as of this option.

An SMTP object has an instance method called **sendmail**, which is typically used to do the work of mailing a message. It takes three parameters –

- The *sender* – A string with the address of the sender.
- The *receivers* – A list of strings, one for each recipient.

## *S11BLH21- Programming in python*

- The *message* – A message as a string formatted as specified in the various RFCs.

### *Example*

Here is a simple way to send one e-mail using Python script. Try it once –

```
import smtplib

sender = 'from@fromdomain.com'

receivers = ['to@todomain.com']

message = """From: From Person from@fromdomain.com

To: To Person to@todomain.com

Subject: SMTP e-mail test

This is a test e-mail message.

"""

try:

    smtpObj = smtplib.SMTP('localhost')

    smtpObj.sendmail(sender, receivers, message)

    print "Successfully sent email"

except SMTPException:
```

Here, you have placed a basic e-mail in message, using a triple quote, taking care to format the headers correctly. An e-mail requires a **From**, **To**, and **Subject** header, separated from the body of the e-mail with a blank line.

To send the mail you use *smtpObj* to connect to the SMTP server on the local machine and then use the *sendmail* method along with the message, the from address, and the destination address as parameters (even though the from and to addresses are within the e-mail itself, these aren't always used to route mail).

If you are not running an SMTP server on your local machine, you can use *smtplib* client to communicate with a remote SMTP server. Unless you are using a webmail service (such as Hotmail or Yahoo! Mail), your e-mail provider must have provided you with outgoing mail server details that you can supply them, as follows –`smtplib.SMTP('mail.your-domain.com', 25)`

## URL ACCESS

### URL( Uniform REsource Locator)

- `urllib` is the module used for fetching URLs

`urllib` is a Python module that can be used for opening URLs. It defines functions and classes to help in URL actions.

With Python we can also access and retrieve data from the internet like XML, HTML, JSON, etc. We can also use Python to work with this data directly.

#Used to make requests

```
import urllib.request

x= urllib.request.urlopen('https://www.google.com/')

print(x.read())
```